



Co-funded by
the European Union



Spatio-Temporal Linked data tools for the AgRI-food data space

stelar-project.eu

Deliverable 3.1: Schema matching and entity linking (v1)



Author(s)/Organization(s)	George Papadakis (UoA), Konstantinos Nikoletos (UoA), Jakub Maciejewski (UoA), Manolis Koubarakis (UoA), Alexandros Zeakis (ARC)
Contributor(s)	Dimitris Skoutas (ARC)
Lead Beneficiary	UoA
Work Package	WP3
Deliverable type	O — other (software)
Delivery Date (DoA)	M15 (30/11/2023)
Actual Delivery Date	M15 (30/11/2023)
Dissemination Level	Public
Abstract:	D3.1 <i>Schema matching and entity linking (v1)</i> is a direct and first output of T3.1 <i>Schema matching and evolution</i> as well as T3.2 <i>Entity linking</i>. This deliverable has a twofold purpose: (1) to present the STELAR component that focuses on Schema Matching and Entity Linking, namely pyJedAI, and (2) to briefly present the new algorithms and techniques that have been developed and integrated into pyJedA in the context of T3.1 and T3.2. The second version of this deliverable (D3.3) is expected to be produced in M30.

Document Revision History			
Date	Version	Author/Contributor/ Reviewer	Summary of main changes
24/09/2023	0.1	George Papadakis (UoA)	Table of Contents
30/10/2023	0.2	Konstantinos Nikoletos (UoA), Alexandros Zeakis (ARC)	Contributions to Sections 2 and 3
07/11/2023	0.3	George Papadakis (UoA)	Finalized Section 3
14/11/2023	0.4	George Papadakis (UoA)	Section 4
21/11/2023	0.5	George Papadakis (UoA)	Sections 5 and 6
29/11/2023	0.6	George Papadakis (UoA)	Finalized Section 2, added Section 1
30/11/2023	1.0	George Papadakis (UoA), Konstantinos Nikoletos (UoA), Jakub Maciejewski (UoA), Manolis Koubarakis (UoA), Alexandros Zeakis (ARC), Wieger Punter (TU/e)	Final version for submission

STELAR CONSORTIUM			
PARTICIPANT NUMBER	PARTICIPANT ORGANISATION NAME	SHORT NAME	COUNTRY
1	ATHENA - RESEARCH AND INNOVATION CENTER IN INFORMATION, COMMUNICATION AND KNOWLEDGE TECHNOLOGIES	ARC	EL
2	NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS	UoA	EL
3	TECHNISCHE UNIVERSITEIT EINDHOVEN	TU/e	NL
4	UNIVERSITAET DER BUNDESWEHR MUENCHEN	UBM	DE
5	RAPIDMINER GMBH	RapidMiner	DE
6	AGROKNOW IKE	AGROKNOW	EL
7	VISTA GEOWISSENSCHAFTLICHE FERNERKUNDUNG GMBH	VISTA	DE
8	ABACO SPA	ABACO	IT
9	FOODSCALE HUB ENTREPRENEURSHIP AND INNOVATION ASSOCIATION	FSH	RS

LEGAL NOTICE

The information and views set out in this application form are those of the author(s) and do not necessarily reflect the official opinion of the European Union. Neither the European Union institutions and bodies nor any person acting on their behalf may be held responsible for the use which may be made of the information contained therein.



**Co-funded by
the European Union**

Project name: Spatio-TEmporal Linked data tools for the AgRI-food data space

Project acronym: STELAR

Topic: HORIZON-CL4-2021-DATA-01-03

Type of action: HORIZON Innovation Actions

Project starting date: 1 September 2022

Project duration: 36 months

© Stelar Consortium, 2022.

Reproduction is authorized provided the source is acknowledged.

Table of Contents

1.	Introduction.....	6
1.1.	Project Overview	6
1.2.	Deliverable Overview	9
2.	pyJedAI: STELAR’s ER component	11
2.1.	Concept and terminology	11
2.2.	Brief overview of state-of-the-art	12
2.2.1	Schema Matching.....	12
2.2.2	Entity Resolution.....	13
2.2.3	Geospatial Interlinking.....	14
2.3.	pyJedAI in a nutshell	15
3.	New functionalities for Schema (and Ontology) Matching	19
3.1.	Schema Mapping	19
3.2.	Schema Clustering	19
4.	New functionalities for Entity Linking	21
4.1.	New Filtering Algorithms	21
4.1.1	A new paradigm for filtering: NN workflows.....	21
4.1.2	Enhancing Dense NN Workflows.....	23
4.1.3	Enhancing Sparse NN workflows	24
4.2.	New Verification Algorithms.....	25
4.2.1	Enhancing the similarity estimation.....	25
4.2.2	Enhancing the clustering for Clean-Clean ER	26
5.	New functionalities for Geospatial Interlinking.....	28
5.1	pyJedAI-spatial	28
5.2	Supervised Progressive GIA.nt	29
6.	Conclusions and Next Steps.....	31
6.1.	Conclusions.....	31
6.2.	Next steps.....	31
6.3.	Long-term goals	32
	References.....	34

List of Tables

Table 1. Methods per blocking workflow step considered in [33].	21
Table 2. Nearest neighbor workflows considered in [33].	22
Table 3. The language models examined for blocking (and matching) in [33].	24
Table 4. The features defined in [38] for Supervised Progressive GIA.nt.	29

List of Figures

Figure 1. STELAR concept overview.	7
Figure 2. Work Packages and their inter-relations.	8
Figure 3. STELAR's Gantt chart.	9
Figure 4. Example of topological relations satisfied by four geometries.	12
Figure 5. The framework of Progressive Geospatial Interlinking.	14
Figure 6. The position of pyJedAI in the KLMS architecture.	15
Figure 7. The architecture of pyJedAI.	16
Figure 8. The three end-to-end workflows for Entity Resolution implemented by pyJedAI.	17
Figure 9. The steps comprising a nearest neighbor workflow for filtering.	22
Figure 10. The solution space of Geospatial Interlinking algorithms that can be constructed by pyJedAI-spatial.	28

List of Abbreviations and Acronyms	
KLMS	Knowledge Lake Management System
JSON	JavaScript Object Notation
RDF	Resource Description Framework
ER	Entity Resolution
SPARQL	SPARQL Protocol and RDF Query Language
TF-IDF	Term Frequency - Inverse Document Frequency
CSV	Comma Separated Values
WKT	Well-Known Text
MBR	Minimum Bounding Rectangle

1. Introduction

1.1. Project Overview

The STELAR project will design, develop, evaluate, and showcase an innovative Knowledge Lake Management System (**KLMS**) to support and facilitate a holistic approach for FAIR (Findable, Accessible, Interoperable, Reusable) and AI-ready (high-quality, reliably labelled) data. The STELAR **KLMS** will allow to (semi-)automatically turn a raw data lake into a knowledge lake by: (a) enhancing the data lake with a knowledge layer; and (b) developing and integrating a set of data management tools and workflows. The knowledge layer will comprise: (a) a data catalogue that offers automatically enhanced metadata for the raw data assets in the lake; and (b) a knowledge graph that semantically describes and interlinks these data assets using suitable domain ontologies and vocabularies. The provided tools and workflows will offer novel functionalities for: (a) data discovery and quality management; (b) data linking and alignment, and (c) data annotation and synthetic data generation.

The **KLMS** will combine both human-in-the-loop and automatic approaches, allowing to leverage background knowledge of domain experts, while minimizing their involvement. It will increase the automation of finding and selecting relevant data sources, of configuring and tuning the involved data management tools, and of designing, executing, and monitoring end-to-end data processing workflows adapted to different user needs. The **KLMS** will also include specialized tools and functions for geospatial, temporal, and textual data. An organization, such as a data-intensive SME or the operator of a data marketplace, will be able to use the STELAR **KLMS** to increase the readiness of its data assets for use in AI applications and for being shared and exchanged within a common data space. The STELAR **KLMS** will be pilot-tested in diverse, real-world use cases in the agrifood data space, one of the nine data spaces of strategic societal and economic importance identified in the European Strategy for Data.¹

Specifically, the objectives of STELAR are as follows:

- *Objective 1: Facilitate and improve data discovery and reuse.* STELAR will empower users to efficiently and timely discover the right data for their needs by automatically enhancing data descriptions to include: (a) additional and more fine-grained metadata automatically extracted from the content; (b) various quality indicators, including domain-specific ones; and (c) data summaries, which can increase energy efficiency of analytics over large data volumes.
- *Objective 2: Support data linking and interoperability.* STELAR will facilitate data linking and interoperability by using linked data technologies to semantically enrich data descriptions and interlink entities across sources. We will address both schema-level and instance-level matching as well as spatio-temporal data alignment and correlations. Our goal is to: (a) automate the configuration of workflows for schema mapping specification and maintenance, as well as for link and correlation discovery; (b) achieve energy efficiency and scalability when dealing with large entity collections and data volumes; and (c) ensure robustness of the proposed techniques.
- *Objective 3: Improve automation and reliability of data annotation and synthetic data generation.* STELAR will improve both the quality and quantity of input data for downstream AI applications, while minimizing the involvement of domain experts, by: (a) automatically annotating and labelling data of different types (textual, geospatial, temporal); (b) offering mechanisms for explainability and for bias detection and mitigation; and (c) providing a combination of black-box and open-box synthetic data generators to overcome label and data scarcity.
- *Objective 4: Validate, evaluate, and demonstrate the developed tools in real-world use cases.* We will evaluate and showcase how the STELAR **KLMS** deals with identified data management gaps and challenges in the agrifood data space through three diverse, real-world use cases: (a) risk prevention in food supply lines; (b) early crop growth predictions; and (c) timely precision farming interventions.

- *Objective 5: Maximize impact.* We will develop a strategy for sustainability and growth to ensure long-term success of the STELAR outcomes, emphasizing open science practices and open-source tools.

Figure 1 illustrates an overview of the STELAR concept. Assume an organization (System 1) that has a data lake and participates in a data space to exchange and share data with other stakeholders (Systems 2, 3, ..., N). According to the **IDS** Reference Architecture Model,² this sharing happens through a network of communicating IDS Connectors that take care of data sovereignty. In this context, the goal of STELAR is to design, develop, evaluate, and showcase an innovative Knowledge Lake Management System that supports such an organization to (semi-)automatically manage and curate the different types of data assets in its data lake, making them FAIR and AI-ready, and thus turning the raw data lake into a knowledge lake. This will increase the value of the organization's data assets within the data space. Moreover, incoming data obtained from other data space participants can also be processed by the **KLMS**, e.g., to be annotated or interlinked.

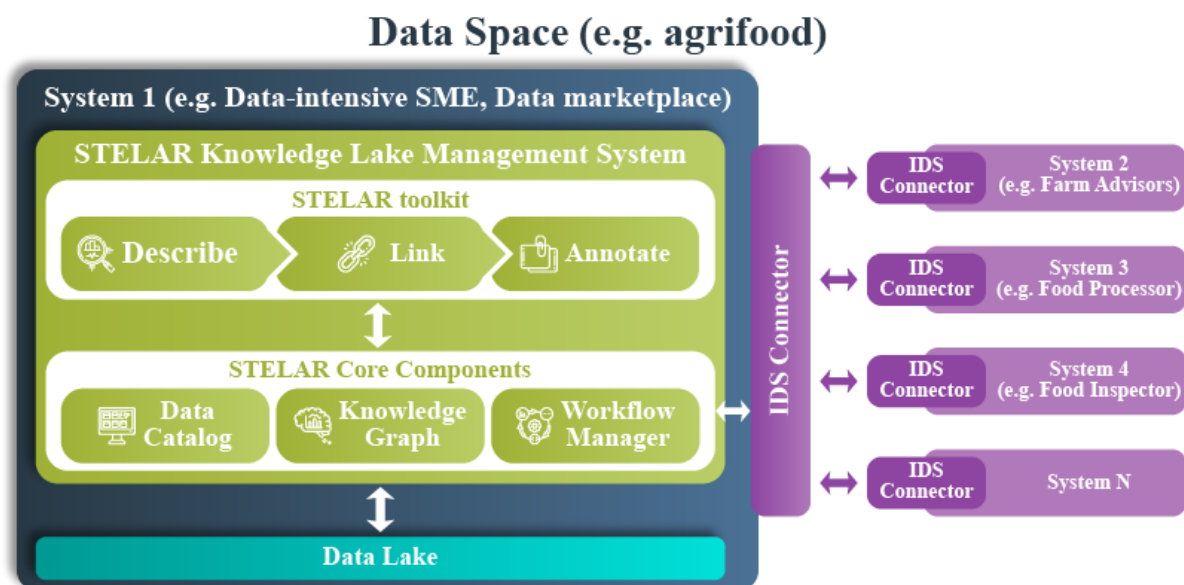


Figure 1. STELAR concept overview.

The STELAR **KLMS** will comprise a set of core components, including a data catalog, a knowledge graph, and a workflow manager, on top of which a modular and extensible set of data management tools will be provided. These tools are grouped into three main categories: (a) data description (i.e., finding the right data); (b) data linking (i.e., combining data from multiple sources); and (c) data annotation (i.e., making data AI-ready).

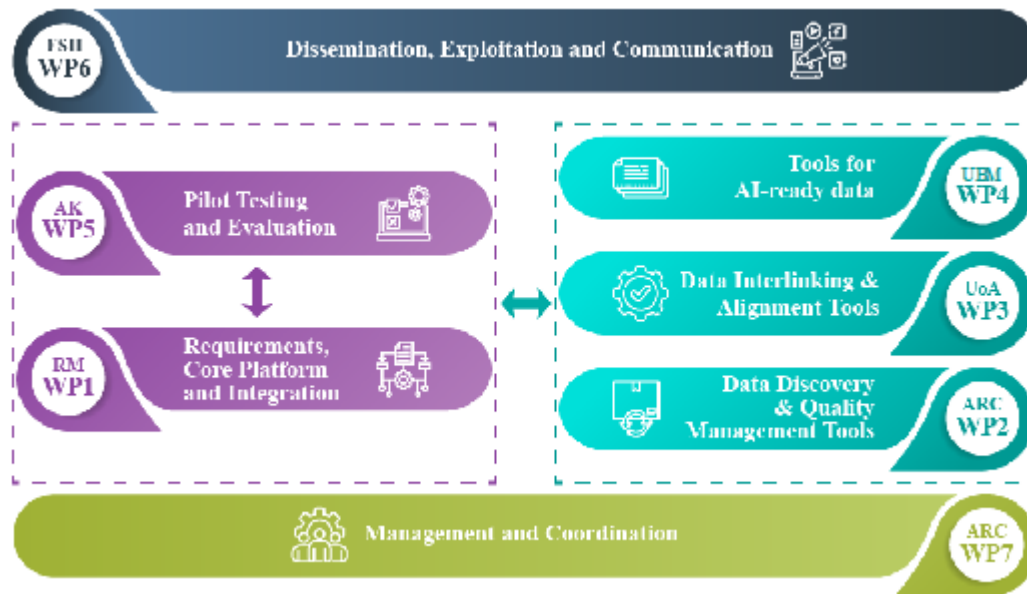


Figure 2. Work Packages and their inter-relations.

The STELAR work plan is scheduled over a period of 36 months and is structured into 7 work packages (WPs), as illustrated in Figure 2 and explained below:

- WPs 1 and 5 focus on requirements and evaluation. WP1 will formalize the use cases, derive the requirements, design the overall architecture, implement the core platform components, and deliver the integrated STELAR **KLMS**. WP5 will conduct the pilot-testing and evaluation, validating and showcasing the STELAR **KLMS** in three pilots covering different stages and types of data sources throughout the food supply chain.
- WPs 2, 3 and 4 will develop the **KLMS** tools. WP2 will develop tools for data discovery and reusability, including data profiling, quality management and summarization. It will also deliver the data catalog and the associated knowledge graph. WP3 will develop tools for automating the creation, validation and maintenance of schema mappings, transformations and entity linkages between different sources, vocabularies, and ontologies. WP3 will also develop tools for spatio-temporal data alignment and correlation. WP4 will develop tools for automating the annotation and labelling of different types of data (textual, geospatial, temporal), for detecting bias and for generating synthetic data.
- WPs 6 and 7 encompass the activities for maximizing the impact of the project and for ensuring smooth coordination and management.

The duration of the Tasks and the timing of Deliverables are depicted in Figure 3.

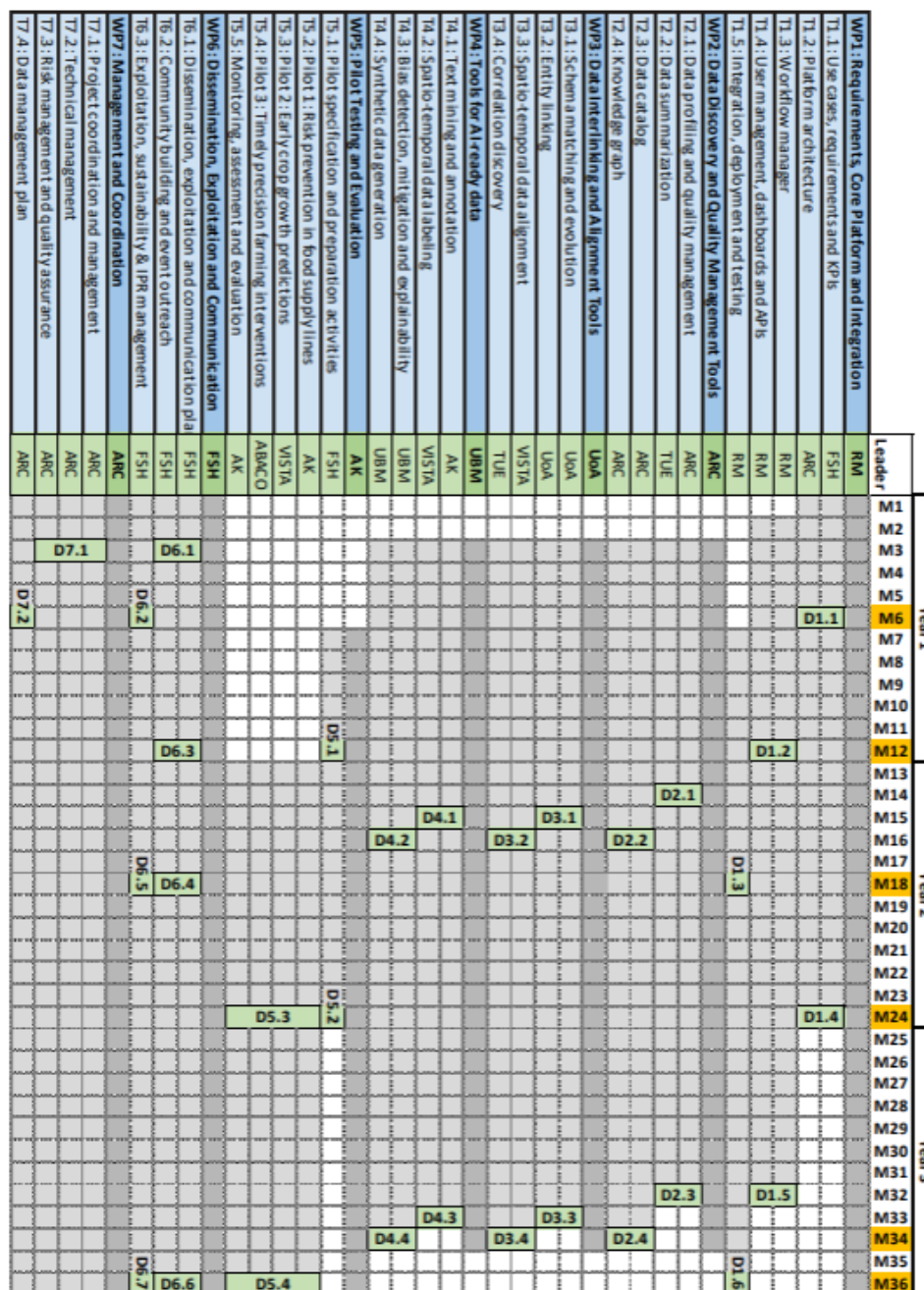


Figure 3. STELAR's Gantt chart

1.2. Deliverable Overview

This deliverable is the output of *Task 3.1 “Schema matching and evolution”* and *Task 3.2 “Entity linking”*, part of *WP3 “Data Interlinking and Alignment Tools”*. As reported in the GA, the objective of WP3 is to tackle interoperability challenges, developing tools for:

- Specifying and maintaining mappings between schemas and aligning terms between ontologies and vocabularies (T3.1).
- Interlinking data from different sources based on semantic or spatio-temporal relations (T3.2).
- Aligning spatial and temporal data coming from different sources (T3.3).
- Detecting correlations between different sources or attributes (T3.4).

The purpose of this deliverable is to present and technically describe all our work in Tasks T3.1 and T3.2 up to M15 of the project. More specifically, this deliverable presents the initial working version **pyJedAI**, the tool that was developed for “*automating the specification of mappings and links between schemas, ontologies, vocabularies, and data instances*”.

More specifically, pyJedAI constitutes an open-source Python package that was developed from scratch in the context of STELAR as a library of state-of-the-art techniques for Entity Resolution. It supports three different end-to-end pipelines, with each one comprising multiple workflow steps. In each workflow step, be it optional or mandatory, pyJedAI provides a series of established methods, adapted so that they implement the same interface. This renders pyJedAI *modular* and *extensible*, facilitating the integration of other Python packages into each workflow step and of custom implementations of existing and new techniques, developed in the context of T3.1 and T3.2. Special emphasis is placed on time efficiency, with pyJedAI boasting a large variety of Filtering techniques for curtailing the quadratic complexity of Entity Resolution. Documentation is also provided for each implemented method and its parameters, thus facilitating users to build end-to-end solutions that are crafted for the task and data at hand. Overall, pyJedAI constitutes an extensive library that allows users for building millions of Entity Resolution workflows with the latest developments in the literature. Its code, including several exemplifying Jupyter notebooks, can be found at the GitHub repository of the STELAR project: <https://github.com/stelar-eu/Schema-Matching-and-Entity-Linking>.

The rest of this report is organized as follows: Section 2 provides preliminary knowledge on the tasks addressed in T3.1 and T3.2, while briefly presenting the state-of-the-art in these fields as well as an overview of pyJedAI’s architecture. Section 3 elaborates on the progress in Task 3.1, i.e., pyJedAI’s new functionalities for Schema Matching. Sections 4 and 5 delve into the progress in Task 3.2: the former describes pyJedAI’s new functionalities on Filtering and Verification (i.e., Matching) in the context of Entity Linking and Entity Resolution more generally, while discussing pyJedAI’s novelties in Geospatial Interlinking. Finally, Section 6 concludes this document along with the main directions for future work in T3.1 and T3.2.

2. pyJedAI: STELAR’s ER component

2.1. Concept and terminology

This section formally defines the main tasks examined in the context of T3.1 and T3.2.

Entity Resolution (ER) is the task of identifying different entity records that represent the same real-world object [12,14]. A brute-force solution needs to consider all pairs of entities, a process that suffers from quadratic time complexity. To ameliorate this issue, the **Filtering-Verification framework** is typically used [13]. The filtering techniques initially reduce the search space to highly similar and, thus, highly likely matches, which are called candidate pairs. These pairs are then analytically processed by a verification approach.

Note that Filtering constitutes an *approximate* procedure that might miss some real matches (i.e., *false negatives*), while retaining *false positives* (i.e., non-matching pairs of entities) [19]. There is a trade-off between these two errors [17]: filtering reduces false negatives to increase recall only by increasing false positives, which reduce precision. Therefore, the goal of Filtering is to strike a good balance between recall and precision, i.e., between false negatives and false positives.

Schema Matching looks for semantic correspondences between structures or models, such as database schemas, XML message formats, and ontologies, identifying different attribute names that describe the same feature (e.g., “profession” and “job” are semantically equivalent) [1][4][11]. The resulting links are crucial in numerous application fields, particularly to support ER tasks, as they facilitate the search for links between two data sources with no known connections between them.

pyJedAI applies ER to data of any *structuredness*, be it structured (like relational database and csv files), semi-structured (like RDF dumps and knowledge graphs), or unstructured (like free text). Any combination of data formats is also possible. **Entity Linking** corresponds to a specific combination, where the goal is to associate entities extracted from free text with entities contained in a knowledge graph or knowledge base [13]. The schema-agnostic settings we consider in STELAR allow pyJedAI to address entity linking through an ER pipeline that matches an unstructured data source, which includes the entity descriptions extracted from text, with a semi-structured one, which involves the RDF statements forming the target knowledge graph.

Geospatial Interlinking focuses on a particular type of entity descriptions, those entailing geo-locations, which are described by geometries [22][23][24]. To facilitate data science applications on top of such descriptions, this task interlinks the geometries with the 9 different types of topological relations that are defined by the DE9IM model¹ (i.e., equals, touches, contains, covers, intersects, within, covered by, crosses, overlaps). As an example, consider the four geometries in Figure 4: g_1 is a LineString that intersects LineString g_2 and touches Polygon g_3 , which contains Polygon g_4 (note that the shaded area corresponds to the intersection of the MBRs of g_1 and g_2 . Its top left corner is used as a reference point to avoid verifying the same pair more than once).

¹ <https://en.wikipedia.org/wiki/DE-9IM>

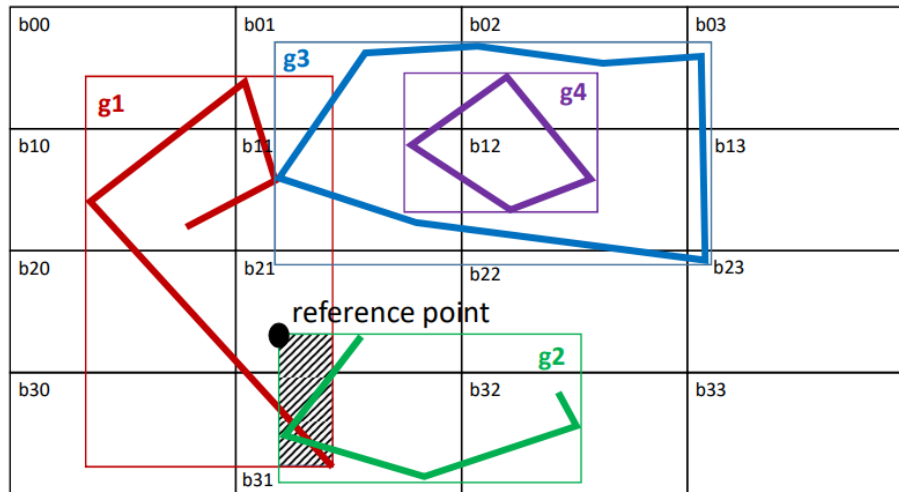


Figure 4. Example of topological relations satisfied by four geometries.

Similar to Entity Resolution, the brute-force solution to Geospatial Interlinking suffers from quadratic time complexity, as it considers all possible pairs of geometries [25]. This leads to poor scalability, which is addressed through the Filtering-Verification framework. The main difference to Entity Resolution is that Geospatial Interlinking is an *exact* task. As a result, Filtering yields no false negatives, i.e., all topologically related geometries are included in the candidate pairs, while Verification yields no false positives, i.e., all topologically related geometries are correctly identified as such.

2.2. Brief overview of state-of-the-art

2.2.1 Schema Matching

There is a plethora of works on schema matching over the last two decades [1]. Nowadays, open-source frameworks like Valentine² [2] comprise the state-of-the-art solutions in the field. More specifically, Valentine offers the following main algorithms for performing schema matching:

1. **Similarity Flooding** [3] constructs a graph representation of the schemas or ontologies to be matched, casting schema matching as a graph matching task. The nodes of the graph represent schema elements, such as attributes, or types, while the edges capture the relationships between these elements. The algorithm starts by initializing similarity scores based on initial mappings or heuristics. It then iteratively propagates similarities through the graph by 'flooding' the network with these scores: each node's similarity score is updated based on the scores of its neighbors, thus capturing the notion that similar nodes are often connected to other similar nodes. This process continues until the similarity scores converge or a specified number of iterations is reached.
2. **Cupid** [4] is a comprehensive approach that involves the four steps: (i) *preprocessing*, which normalizes schema elements by expanding abbreviations and removing special characters to ensure that linguistic comparisons are effective, (ii) *linguistic matching*, which uses a thesaurus or lexical database, like WordNet, to find semantic relationships (synonyms, hypernyms, etc.) between names and descriptions of schema elements, assigning linguistic similarity scores. (iii) *structural matching*, which infers the context and relatedness of elements from the hierarchical structure of the schema (e.g., parent-child relationships), (iv) *similarity aggregation*, which combines linguistic and structural similarities using a weighted average that can be adjusted to prioritize one aspect over the other depending on the context.
3. **COMA** [5] constitutes a flexible approach for combining several complementary techniques, such as linguistic matching based on names and descriptions, structural matching based on schema hierarchies, and constraint-based matching considering keys and cardinalities. These matchers can be executed in parallel or sequentially, and their results can be aggregated using different strategies,

² <https://delftdata.github.io/valentine/>

such as weighted averages or maximum selection. The end result is a consolidated similarity matrix that reflects the combined evidence from all applied matchers.

4. **EmbDI** [6] uses a compact graph-based representation that captures the relationships between the elements of the given schema, facilitating the extraction of sentences that effectively describe the similarity between elements. The resulting sentences are used to learn embeddings that are crafted for the data at hand, using optimizations to improve their quality.
5. **SemProp** [7] is composite approach that combines syntactic with semantic similarity. First, it applies the Semantic Matcher, which leverages the semantic similarity between schema elements, as it is determined by pre-trained word embeddings. This similarity is used to yield *coherent groups*, i.e., clusters of words for which the average cosine similarity of their embedding vectors is higher than a specific threshold. Pairs of attributes that cannot be matched in this way, are associated with syntactic similarities based on the textual similarity of their instances (i.e., attribute values) and attribute names.

Emphasizing time efficiency, **LogMap** [8] is an ontology matching approach that scales to tens (or even hundreds) of thousands of classes. To ensure high effectiveness, it incorporates sophisticated reasoning and repair techniques to minimize the number of logical inconsistencies. It also supports user interventions during the matching process through its graphical user interface.

Finally, **DeepOnto** [9] offers a library of ontology engineering methods that are based on deep learning. Among them, **BERTMap** [10] is a family of approaches that, as its name suggests, leverages the context-aware, semantic similarities offered by the BERT language model. It starts by predicting mappings through a classifier that leverages BERT fine-tuned on text corpora extracted from ontologies. The initial results are then refined through extension and repair based on the ontology structure and logic.

2.2.2 Entity Resolution

There is a plethora of works on Entity Resolution over the last decades. They are summarized in books [10] [11] [12] [13], tutorials [14][15][16] and surveys [17][18][19]. Delving into this literature, is beyond the scope of this deliverable. To facilitate the understanding of the works, we follow the taxonomy defined in [13], which categorizes the main existing works into four generations according to the data challenges they face.

The **first generation** focuses on *Veracity*, i.e., their goal is to transform the input entity profiles into an accurate set of entities that is as close as possible to the corresponding real-world objects. Their input typically comprises structured data that are homogeneous or involve low levels of schema diversity. In other words, this generation primarily targets noise in the attribute values of profiles, operating in a schema-aware fashion.

The **second generation** addresses both *Veracity* and *Volume*, as the number of given entity profiles increases to (tens of) millions. The quality of input data remains the same, involving relatively homogeneous structured data. Therefore, the performance target is to combine accurate results with high scalability.

The **third generation** addresses *Veracity*, *Volume* and *Variety*, as the given entity profiles exhibit high levels of schema and value heterogeneity (i.e., there are high levels of noise both in attribute names and attribute values). Moreover, the data format has changed from typically relational data to semi-structured (RDF) and even unstructured (free text) as well as to repositories of data of any structuredness (i.e., data lakes). Methods of this category emphasize scalability in the context of loose schema information or even under a schema-agnostic functionality, that considers all attribute values (e.g., in blocking and matching), regardless of the associated attribute names and their relations.

The **fourth generation** addresses *Veracity*, *Volume*, *Variety* and *Velocity*, as there are time constraints with respect to the ER running time, while the volume of input data increases continuously with time. When the time constraints are relatively loose, in the order of minutes or even hours, pay-as-you-go techniques are required to produce partial solutions that are extended as the available time increases. When the time constraints are strict, real-time techniques are used instead.

Tasks 3.1 and 3.2 have examined methods that cover the first and third generation. Methods for the second generation, leveraging multi-core and/or massive parallelization, will be integrated before Milestone 3 on M18. Methods for the fourth generation will be incorporated in the second half of the project.

Note that there are several open-source tools that implement established algorithms of ER, with most of them offering techniques of a particular generation. For example, Magellan³ is crafted for Generation 1, while Silk⁴ and LIMES⁵ focus on Generation 3. As explained in [13][18], none of them covers two or more generations, with the sole exception of JedAI (see below for more details).

2.2.3 Geospatial Interlinking

As explained above, all works on Geospatial Interlinking follow the Filtering-Verification framework to significantly reduce the quadratic complexity of the brute-force approach.

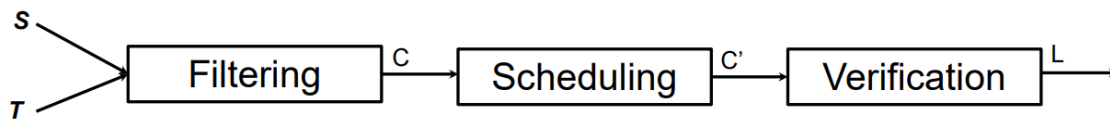


Figure 5. The framework of Progressive Geospatial Interlinking.

Silk-spatial [20], the first relevant work in the literature, divides the Earth surface into tiles with user-specified dimensions, during Filtering. During Verification, it examines the candidate pairs inside every tile in parallel using Apache Hadoop⁶.

RADON [21] turns Filtering into a data-driven process, where the dimensions of the Grid index depend on the input geometries (i.e., on their average width and length). Every geometry is then placed in all times intersecting its Minimum Bounding Rectangle (MBR). For this reason, its Verification keeps in main-memory all pairs of geometries that have already been verified to avoid repeating their verifications in different tiles.

RADON2 [22] improves RADON's verification so that it simultaneously extracts all topological relations per geometry pair, instead of repeating the Filtering-Verification scheme for each topological relation of interest.

stLD [23] goes beyond RADON in four ways:

- (i) It supports various established indices for Filtering, such as R-Tree, and a hierarchical grid.
- (ii) It leverages massive parallelization on top of Apache Flink⁷.
- (iii) Instead of loading both the source and the target datasets in main memory, it loads only the former one, while using the latter as a stream of geometries that is read on-the-fly (from the disk).
- (iv) During Filtering, MaskLink skips the examination of entire tiles per target geometry t if t is contained in the space left empty by all source geometries in the tile.

GIA.nt [24] combines the advantages of all the above works. Its Filtering loads only the smallest dataset D_s in main memory and indexes it with an Equigrid, whose dimensions correspond to the average length and width in D_s . Its Verification reads the largest dataset from the disk, one geometry at a time, retrieving the candidate pairs per geometry from the Equigrid and verifying those with intersecting MBRs. Both processes are parallelized on top of Apache Spark⁸.

Progressive GIA.nt [24] turns the above *exact*, *batch* algorithms into an *approximate*, *pay-as-you-go* process, whose goal is to maximize the topologically related pairs within a limited available budget of

³ https://github.com/anhaidgroup/py_entitymatching

⁴ <http://silkframework.org>

⁵ <https://github.com/dice-group/LIMES>

⁶ <http://hadoop.apache.org>

⁷ <https://flink.apache.org>

⁸ <https://spark.apache.org>

computational time or resources (e.g., in the context of cloud-based applications, where certain functionalities like the AWS Lambda functions charge whenever used). To this end, it extends the Filtering-Verification framework with an intermediate step, called *Scheduling*, as shown in Figure 1. Scheduling first associates every candidate pair of geometries generated by Filtering with a weight that is proportional to the likelihood that they are topologically related. Then, it sorts all candidate pairs in decreasing weight. Verification processes the next top-weighted pair until the available budget is consumed.

Progressive RADON [24] first orders the tiles of the Equigrad according to the number of geometry pairs they include and then applies Progressive GIA.nt inside every tile.

Dynamic Progressive GIA.nt [25] replaces Scheduling, which produces a static processing order, with Dynamic Scheduling, where the weight of candidate geometry pairs is updated whenever a new pair of topologically related pairs is detected.

2.3. pyJedAI in a nutshell

In the context of Tasks 3.1 and 3.2, we have developed **pyJedAI** [32], an open-source library that leverages Python’s data science ecosystem to build powerful end-to-end workflows for these tasks. As depicted in Figure 6, pyJedAI is part of the *KLMS Tools* module of the STELAR KLMS, under the more generic name *Interlinking Tools*, so that it can be invoked by the workflow execution engines.

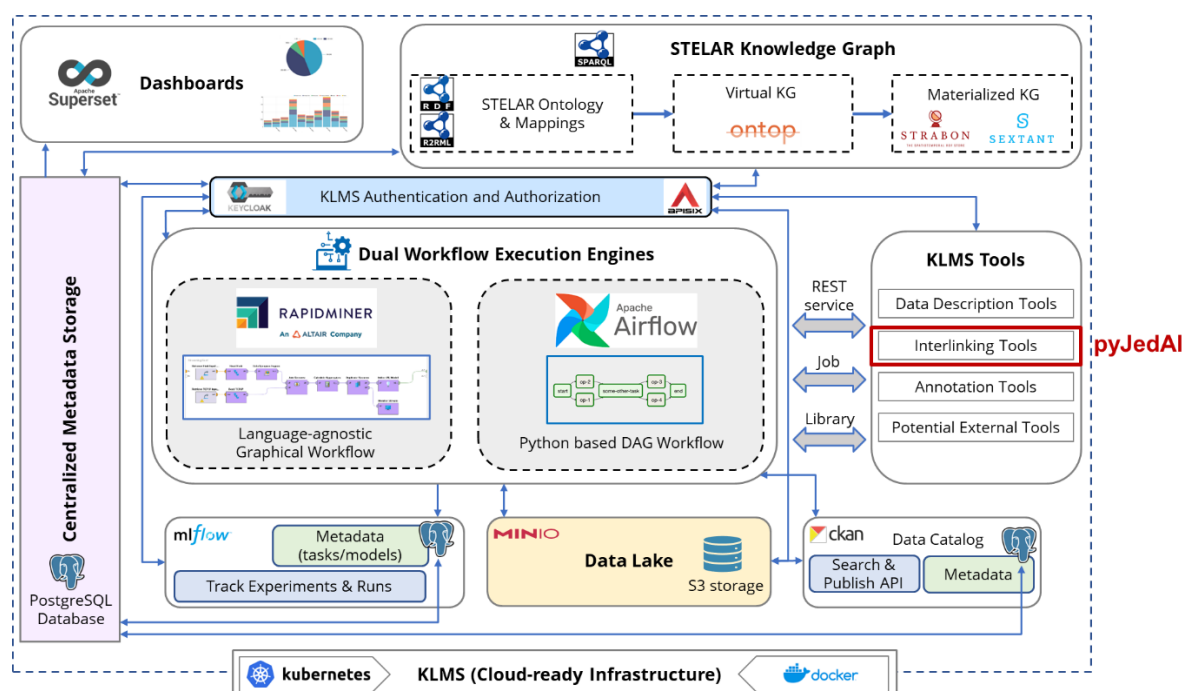


Figure 6. The position of pyJedAI in the KLMS architecture.

We have publicly released the source code of pyJedAI at <https://github.com/AI-team-UoA/pyJedAI> under Apache License V2.0, which supports both academic and commercial applications. pyJedAI has also been uploaded to the PyPi and, thus, it can be easily installed with the following command:

```
pip install pyjedai
```

pyJedAI builds on an older open-source system with a similar purpose, called JedAI⁹. However, JedAI constitutes an isolated system, implemented in Java, which cannot be easily extended with existing

⁹ <https://github.com/scify/JedAIToolkit>

state-of-the-art techniques from other domains, like Deep Learning and Natural Language Processing (NLP). pyJedAI goes beyond JedAI by implementing the same methods as JedAI in Python, so that it is capable of combining them with any package from Python’s data science ecosystem. As a result, pyJedAI extends JedAI with a series of new algorithms, as explained below, in Sections 3, 4 and 5 in more detail.

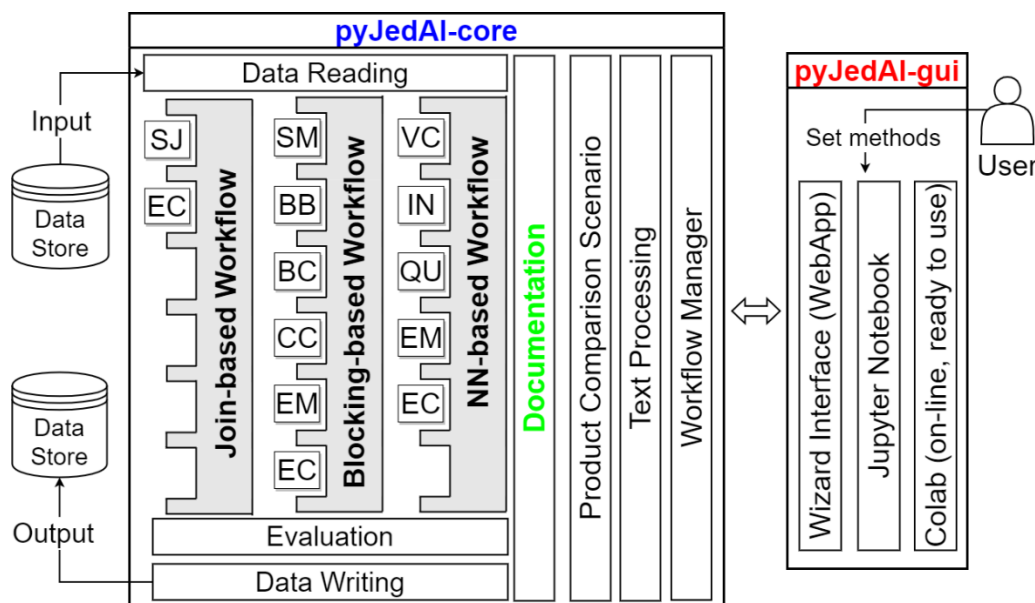


Figure 7. The architecture of pyJedAI.

Figure 7 outlines the architecture and the main functionalities of pyJedAI. For the I/O operations, pyJedAI leverages the pandas library¹⁰ for reading and storing data in a large variety of formats (CSV, JSON and MS Excel files, relational databases and SPARQL endpoints). RDF dumps are also supported through rdfpandas¹¹. For example, a Clean-Clean ER dataset can be read and transformed into the pyJedAI’s internal data structure with the following commands:

```
import pandas as pd
import pyjedai
from pyjedai.datamodel import Data

d1 = pd.read_csv("../data/ccer/D2/abt.csv", sep='|', engine='python',
                 na_filter=False)
d2 = pd.read_csv("../data/ccer/D2/buy.csv", sep='|', engine='python',
                 na_filter=False)
gt = pd.read_csv("../data/ccer/D2/gt.csv", sep='|', engine='python')

data = Data(dataset_1=d1, id_column_name_1='id', dataset_2=d2,
            id_column_name_2='id', ground_truth=gt)
```

After the data reading step, an end-to-end workflow should be created and configured by the user to process the given data. pyJedAI implements three end-to-end workflows for Entity Resolution, as shown in Figure 7. Each workflow consists of multiple components, which are shown in more detail in Figure 8. In each component, pyJedAI offers a variety of the corresponding state-of-the-art algorithms

¹⁰ <https://pandas.pydata.org>

¹¹ <https://github.com/cadmiumkitty/rdfpandas/tree/master>

in the literature, which are implemented under a unified interface that facilitates the composition of end-to-end pipelines by the user.

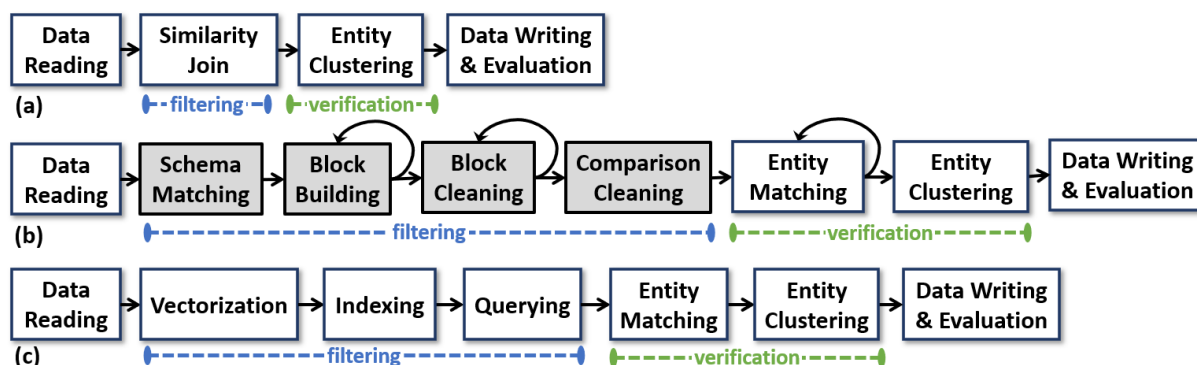


Figure 8. The three end-to-end workflows for Entity Resolution implemented by pyJedAI.

More specifically, the *Join-based workflow* in Figure 8(a) encapsulates ER techniques of the first generation, which exclusively target Veracity. Therefore, it uses the values of a particular record attribute. It is thus suitable for datasets with clean and aligned schemata that involve at least one highly distinctive attribute with low levels of noise (e.g., relational databases). In these settings, it suffices to consider as candidates those pairs of records that exhibit a very high similarity. For Filtering, it employs a *similarity join* algorithm, which quickly estimates the pairs of entity profiles with a similarity higher than a threshold. The resulting pairs form a **similarity graph**, where the nodes correspond to entity profiles and the edges connect candidate pairs, with their weights corresponding to the similarity estimated by the join algorithm. For Verification, an *entity clustering* algorithm partitions the similarity graph into equivalence clusters such that every cluster contains all entity profiles corresponding to the same real-world object and there is a single equivalence cluster per real-world object.

The *Blocking-based workflow* in Figure 8(b) is crafted for the third generation of ER techniques, which target Veracity, Volume and Variety, i.e., high levels of heterogeneity and noise in both attribute names and values. For this reason, it typically considers all attribute values in the input entity profiles, thus being more suitable for datasets with heterogeneous schemata. After data reading, it applies *schema matching* to align similar attributes. The next three steps apply Filtering, by separating entities into blocks (i.e., groups) such that the records within each block are candidate matches, while the records of different blocks are dissimilar. The first filtering component is *block building*, which creates a separate block for each token in the attribute values or parts of it, using for example q-grams or suffixes (e.g., all entity profiles with a particular q-gram or suffix in their attribute values are placed in the corresponding block and are considered as candidates with all other co-occurring records). The number of candidates generated in this way is initially high but is drastically reduced with *block* and *comparison cleaning*: the former discards blocks with an excessively high number of candidates, as they probably correspond to stop-words, while the latter removes the repeated pairs, which appear in multiple blocks. Comparison cleaning also weighs each candidate pair according to its co-occurrence patterns so that only the top-weighted ones are retained and fed to Verification. This involves *entity matching*, which computes the syntactic similarity between all candidate pairs and builds a similarity graph, and *entity clustering*, is the same as in the Join-based workflow.

Unlike the above workflows, which rely on *syntactic* similarities, i.e., on the common character or token n-grams between entity profiles, the *NN-based workflow* in Figure 8(c) leverages *semantic* similarities, i.e., the proximity of the embedding vectors defined by pre-trained language models. To this end, the input entities are converted into dense embedding vectors by the selected pre-trained language model during the *vectorization* step. Next, the vectors of the largest entity collection are *indexed* using one of

the state-of-the-art tools for approximate nearest neighbor search, such as FAISS¹². The resulting index is *queried* with every embedding vector of the smallest dataset to retrieve the k nearest neighbors per query. This set of neighbors forms the candidate pairs that are further processed by *entity matching*, which computes the similarity of the respective embedding vectors and builds the corresponding similarity graph. The graph is then fed to *entity clustering*, which produces the final equivalence clusters.

Note that the NN-based workflow is exclusively provided by pyJedAI, while the other two are also provided by Java JedAI. Even in these cases, though, pyJedAI extends Java JedAI with more advanced methods per component/workflow step, as explained in Sections 3, 4 and 5 in more detail. Note also that the NN-based workflow belongs to both the first and the third generation of ER techniques. In the first case, where the only challenge is Veracity, it can be applied in a schema-aware manner that exclusively considers the values of specific attributes. In the latter case, where the challenges include Veracity, Volume and Variety, it can be applied in a schema-agnostic manner that considers all attribute values per entity profile.

Regardless of the selected workflow, the detected duplicate pairs can be exported and stored in any of the supported data formats, from structured ones (e.g., CSV files and relational databases) to semi-structured ones (SPARQL endpoints and RDF dumps).

Overall, pyJedAI supports millions of different end-to-end pipelines for Entity Resolution. To facilitate users of any level of experience, from novice to expert ones, to build these pipelines, pyJedAI offers three intuitive user interfaces: a Web app with a wizard-like interface¹³, a Colab notebook¹⁴ that is online and ready to be used as well as Jupyter notebooks¹⁵. All user interfaces leverage the detailed documentation that is provided by each implemented algorithm, which explains its main functionality, its configuration parameters as well as the reasonable values per parameter.

¹² <https://faiss.ai>

¹³ <https://github.com/AI-team-UoA/JedAI-WebApp>

¹⁴ https://colab.research.google.com/drive/1VB_DfIT3eLXhlg6vGZSCWrJKc7AcLIpA

¹⁵ See here for examples: <https://github.com/stelar-eu/Schema-Matching-and-Entity-Linking/tree/main>

3. New functionalities for Schema (and Ontology) Matching

In this section, we elaborate on how pyJedAI goes beyond the state-of-the-art in Schema Matching in the context of Task 3.1. More specifically, two types of algorithms have been incorporated into the Schema Matching component of pyJedAI. Each one is described in a separate subsection.

3.1. Schema Mapping

This type of algorithms is suitable for structured data with low levels of heterogeneity, e.g., when integrating a relational database with a CSV file. Therefore, emphasis is placed on effectiveness. To this end, pyJedAI has incorporated Valentine as a plugin so that all state-of-the-art algorithms mentioned in Section 2.2 are available in pyJedAI. pyJedAI has also incorporated the latest works on Schema Matching, which rely on the combination of Deep Learning and language models, as implemented by DeepOnto, which can also be used through pyJedAI.

As an example, pyJedAI can match the schemata of two structure datasets with Valentine’s Cupid algorithm with the following commands:

```
from pyjedai.schema_matching import ValentineMethodBuilder,
ValentineSchemaMatching

sm = ValentineSchemaMatching(ValentineMethodBuilder.cupid_matcher())
sm.process(data)
sm.get_matches()
```

Note that all these methods supported by pyJedAI go beyond Java JedAI, which does not include any relevant techniques. Note also that pyJedAI goes beyond Valentine and DeepOnto by combining their results with any bipartite graph clustering techniques described in Section 4.2.2.

More specifically, each algorithm implemented by Valentine and DeepOnto produces a set of mappings, i.e., pairs of attribute names, with each one associated with a similarity score that indicates the strength of the match between them. These mappings are then thresholded to select the most likely correspondences. In the retained pairs, though, it is common that a single attribute name from dataset D_1 is associated with multiple attribute names from D_2 and/or vice versa. In cases where there is 1-1 correspondence between the two datasets, pyJedAI transforms the retained mappings into a bipartite graph matching task and applies a clustering algorithm to extract the correct attribute pairs. For example, Unique Mapping Clustering orders the pairs in decreasing similarity score and in each iteration, it treats as a correct mapping the top-weighted one as long as none of its constituent attribute has already been mapped to another attribute.

3.2. Schema Clustering

For datasets with very large schemata (e.g., ontologies) and levels of heterogeneity, pyJedAI implements a novel approach that emphasizes scalability and time efficiency over effectiveness. The scope moves from identifying semantically equivalent attribute names across different datasets to clustering together attribute names so that the performance of blocking and entity matching is improved. This idea was first successfully applied to block building by Attribute Clustering Blocking, which partitioned attribute names according to the value of their similarities and then applied Standard Blocking to the values of each cluster independently of all other attribute clusters; as a result, precision raised to a significant extent for a negligible decrease in recall.

Attribute Clustering is implemented in Java JedAI, but pyJedAI generalizes this approach in the following ways:

- It can incorporate any similarity function between attribute names and/or attribute values that is already supported by pyJedAI’s Verification module (i.e., Entity Matching). This includes

syntactic similarities like those based on character or token n-grams with TF-IDF weights as well as semantic similarities like the language models listed in Table 3.

- Special care is taken to accelerate the run-time through indexing (e.g., through the NN workflow in Figure 9, in case semantic similarities are used).
- It can be combined with any clustering algorithm implemented by pyJedAI to partition the highly similar attributes into disjoint clusters. This includes both clustering algorithms for Dirty ER [27] and Clean-Clean ER (see Section 4.2.2).

In more detail, Schema Clustering operates as follows:

1. It creates an attribute graph, where the nodes correspond to different attributes.
2. Edges are added between the nodes with a non-zero similarity (based on their names, values or both).
3. A clustering algorithm is applied partition the graph into disjoint attribute clusters so that the values of each cluster are treated independently of all others in blocking and/or matching.
4. All attributes that are isolated, i.e., they are not included in any cluster with at least two attributes (from different datasets) are placed in the same, glue cluster.

Note that Schema Clustering is applicable even in the case of a single input dataset as long as its schema is large enough to be partitioned into multiple disjoint clusters. Note also that only the attributes are split among different, non-overlapping clusters, while a single every entity profile might participate in multiple attribute clusters.

4. New functionalities for Entity Linking

In this section, we elaborate on how pyJedAI goes beyond the state-of-the-art in Entity Linking in the context of Task 3.2.

4.1. New Filtering Algorithms

We now elaborate on the new algorithms for blocking that were added to pyJedAI in the context of three publications.

4.1.1 A new paradigm for filtering: NN workflows

This work [33] implemented and evaluated the relative performance for two types of filtering workflows under the schema-agnostic settings (i.e., every entity profile is represented a sentence that concatenates of all its attribute values, completely disregarding all attribute names).

- 1) The **blocking workflows**, which consist of three steps, as depicted in Figure 8(b):
 - a. *Block Building*, which extracts blocking keys from entity profiles and clusters together entities with identical or similar keys.
 - b. *Block Cleaning*, which targets entire blocks, removing the oversized ones, which involve numerous entity pairs/comparisons, the vast majority of which is repeated, i.e., already included in another block.
 - c. *Comparison Cleaning*, which targets individual pairs, removing all repeated comparisons among the overlapping blocks as well as part of the superfluous comparisons, which involve non-matching entities.

Table 1 lists the methods that were considered per workflow step in [32]. Note that Block Cleaning is optional, but if applied, any combination of the two available methods can be used. The other two steps are mandatory, but only one of the available techniques can be used. Comparison Cleaning is formed by a combination of a pruning algorithm with a weighting scheme.

Table 1. Methods per blocking workflow step considered in [33].

Block Building	Blocking Cleaning	Comparison Cleaning	
Standard Blocking	Block Purging	Weighted Edge Pruning (WEP)	Aggregate Reciprocal Cardinality Scheme (ARCS)
Q-Grams Blocking	Block Filtering	Weighted Node Pruning (WNP)	Common Blocks Scheme (CBS)
Extended Q-Grams Blocking		Cardinality Edge Pruning (CEP)	Extended Common Blocks Scheme (ECBS)
Suffix Arrays Blocking		Cardinality Node Pruning (CNP)	Jaccard Similarity (JS)
Extended Suffix Arrays Blocking		Reciprocal Weighted Node Pruning (RWNP)	Extended Jaccard Similarity (JS)
		Reciprocal Cardinality Node Pruning (RCNP)	Pearson χ^2 Test
		BLAST	

All these methods were already implemented in Java JedAI, and were reimplemented in pyJedAI. The latter ensures that all blocking workflows can be easily applied to a specific dataset with few commands. For example, the workflow formed by the methods in the first row of Table 1 are run through the following commands:


```
from pyjedai.block_building import StandardBlocking
from pyjedai.block_cleaning import BlockPurging
from pyjedai.comparison_cleaning import WeightedEdgePruning

bb = StandardBlocking()
blocks=bb.build_blocks(data,attributes_1=['name'], attributes_2=['name'])

bf = BlockFiltering(ratio=0.8)
filtered_blocks=bf.process(cleaned_blocks,data,tqdm_disable=False)

mb = WeightedEdgePruning(weighting_scheme='ARCS')
candidate_pairs_blocks=mb.process(filtered_blocks,data,tqdm_disable=True)
```

- 2) The **nearest-neighbor (NN) workflows**, which involve the steps presented in the Figure 9:
- Clean*, which is an optional step that removes stop-words and performs stemming.
 - Embed*, which is another optional step that vectorizes each entity profile using a language model, transforming it into a dense multi-dimensional vector.
 - Index*, which indexes one of the input data sources using an optimized data structure so that the most similar entities to a query can be efficiently retrieved.
 - Query*, which uses every entity profile of the other input data source as a request for retrieving its k-nearest neighbors from the other data source. The nearest neighbors of all input entities form the resulting set of candidate pairs.

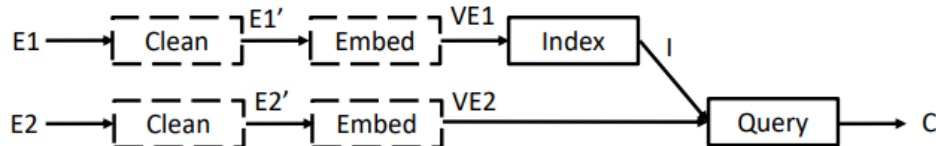


Figure 9. The steps comprising a nearest neighbor workflow for filtering.

Table 2 lists the methods NN workflows that were considered in [32]. The Sparse NN workflows skip the Embed step, as they rely on traditional term vectors with term frequency or TF-IDF scores as weights, yielding sparse multi-dimensional vectors. This way, they capture the syntactic similarity of entity profiles. In contrast, the Dense NN Workflows capture their semantic similarity, leveraging the pre-trained fastText embeddings to transform every entity profile into a 300-dimensional numeric vector.

Table 2. Nearest neighbor workflows considered in [33].

Sparse NN workflows	Dense NN workflows
Similarity Join	Hyperplane LSH
k-NN Join	Cross-Polytope LSH
MinHash LSH	FAISS
	SCANN
	DeepBlocker

Among these methods, only the similarity join was part of Java JedAI. The rest were added to pyJedAI for the first time. We used custom implementations for the Sparse NN workflows, while for the Dense ones relied on existing python packages (FALCONN¹⁶ for Hyperplane and Cross-Polytope LSH and

¹⁶ <https://github.com/FALCONN-LIB/FALCONN>

the homonymous packages for the rest of the workflows: Facebook’s FAISS¹⁷, Google’s SCANN¹⁸ and DeepBlocker¹⁹). Pre-trained fastText embeddings are provided by Gensim²⁰.

Using pyJedAI, all methods can be easily applied to a specific dataset. For example, FAISS can be run through the following commands:

```
from pyjedai.vector_based_blocking import EmbeddingsNNBlockBuilding

emb = EmbeddingsNNBlockBuilding(vectorizer='fasttext',
                                similarity_search='faiss')
blocks, g = emb.build_blocks(data, top_k=5, similarity_distance='euclidean')
```

Comparing the performance of these two fundamentally different filtering approaches is a non-trivial task. To overcome this obstacle, we considered the highest precision that is achieved by each approach for a recall at least 90%.

The experimental analysis was performed on 10 established real-world datasets crafted for Clean-Clean Entity Resolution. Each involved several thousand entities from four different domains: restaurant names, product descriptions, bibliographic data and movie information. Exhaustive grid search was performed on each dataset over a set of reasonable configuration parameters per method to identify the best possible performance per dataset.

The scalability analysis involved 7 synthetic datasets with census data (i.e., names and addresses), whose size increases from 10,000 and 50,000 entity profiles to 100, 200, 300 thousand profiles and to 1 and 2 millions. Due to computational restrictions, the fine-tuning of each method was performed on the smallest datasets and the resulting configuration was applied to the rest of the datasets.

The experimental results suggest that:

- (i) The blocking workflows based on the parameter-free Standard Blocking excel in effectiveness.
- (ii) The kNN-Join excels in robustness and usability, as a straightforward configuration consistently achieves very high performance in most datasets.
- (iii) FAISS excels in scalability, when performing approximate NN search rather than exact one.

4.1.2 Enhancing Dense NN Workflows

The superiority of blocking workflows mainly stems from their flexibility: every method in a blocking workflow involves at least one configuration parameter with numerous reasonable values, which increases the configuration space exponentially. This way, Standard Blocking involves 3,440 different configuration per dataset, which is the minimum one among blocking workflows (the maximum configuration space corresponds to Extended Q-Grams Blocking with 68,800 configurations).

In contrast, the top-performing NN-workflow in [32], FAISS, is configured by: (i) the absence or presence of cleaning, (ii) the data source that will be indexed (leaving the other one for querying), and (iii) the number of nearest neighbors per query.

To enhance the flexibility of FAISS, we extended its configuration space with the language models that can be used for embedding entity profiles into dense multi-dimensional vectors in [33]. More specifically, we considered the state-of-the-art language models in their pre-trained form, because blocking typically constitutes an unsupervised approach requiring no labelled instances.

In this context [34], we considered 12 different models that are distinguished into the three categories listed in Table 3:

¹⁷ <https://faiss.ai>

¹⁸ <https://github.com/google-research/google-research/tree/master/scann>

¹⁹ <https://github.com/qcri/DeepBlocker>

²⁰ <https://radimrehurek.com/gensim>

Table 3. The language models examined for blocking (and matching) in [33].

Static Models	Bert-based Models	SentenceBERT models
Word2Vec	BERT	S-MPNet
FastText	AlBERT	S-GTR-T5
GloVe	RoBERTa	S-DistilRoBERTa
	DistilBERT	S-MiniLM
	XLNet	

The static models are context-agnostic, in the sense that they always convert every token into the same embedding vector. In contrast, the BERT-based ones are context-aware, producing different word embeddings depending on the meaning in each sentence (e.g., they yield a specific vector for a bank as a financial institution and a different one for bank as the side of a river). The SentenceBERT models extend BERT models by directly producing embeddings for entire sentences, rather than individual words.

Neither of these models was part of Java JedAI. They were added to pyJedAI through Gensim, which provides Word2Vec and FastText, and Hugging Face²¹, which provides the rest of the models. All models can be easily applied to any dataset with few commands. For example, a robust and usually quite effective configuration for S-GTR-T5 can be run as follows:

```
from pyjedai.vector_based_blocking import EmbeddingsNNBlockBuilding

emb = EmbeddingsNNBlockBuilding(vectorizer='st5', similarity_search='faiss')
blocks, g = emb.build_blocks(data, top_k=10, similarity_distance='euclidean')
```

In fact, our extensive experimental analyses indicate that this configuration is consistently the top-performing one among language models, while outperforming the blocking workflows in most cases, too.

4.1.3 Enhancing Sparse NN workflows

While [34] improves the Dense NN workflows considered in [33], [35] improves the Sparse NN workflows and in particular the Similarity Join workflow that is considered in [33]. More specifically, TokenJoin improves Set Similarity Joins, which represent each entity profile as a set of tokens (e.g., the words in a product description or the words in a movie title) in their effort to find entities that share many common tokens. In many cases, though, these tokens cannot match exactly, but might match approximately. In these cases, Set Similarity Joins identify candidate pairs by identifying the most similar pairs of words between two entity profiles. This requires computing all similarities between all words describing the two entities, forming a bipartite graph and then applying a Maximum Weighted Bipartite Matching to find the final similarity score between the two entities.

To this end, TokenJoin applies token-based filtering, meaning that instead of using expensive element-based filters, it utilizes tokens and their Upper Bounds to efficiently prune false candidates. It is implemented both in Java²² and Python²³.

To install pyTokenJoin, run:

```
pip install pytokenjoin
```

A simple example in Python lies below:

- To load data, read a pandas DataFrame with a column as id and a textual column that will be transformed into a list of tokens.

²¹ <https://huggingface.co>

²² <https://github.com/alexZeakis/TokenJoin>

²³ <https://github.com/alexZeakis/pyTokenJoin>

```
import pandas as pd
```

```
file = '../data/yelp_sample.csv'
df = pd.read_csv(file, header=None, nrows=1000)
df = df.reset_index(drop=False)
df.columns = ['id', 'text']
df.text = df.text.apply(lambda x: x.split(';'))
df.text = df.text.apply(lambda x: list(set(x)))
```

- To run TokenJoin with a threshold δ :

```
from pytokenjoin.jaccard.join_delta import JaccardTokenJoin

out_df = JaccardTokenJoin().tokenjoin_self(df, id='id', join='text',
posFilter=True, jointFilter=True, delta=0.7)
```

- To run TokenJoin with top-k results:

```
from pytokenjoin.jaccard.join_topk import JaccardTokenJoin

out_df=JaccardTokenJoin().tokenjoin_self(df,id='id',join='text', k=100)
```

Apart from self-joins with a single DataFrame, pyTokenJoin also supports foreign joins with two DataFrames. More examples can be found in the Github repository.

4.2 New Verification Algorithms

As explained above, pyJedAI focuses on unsupervised entity matching, with the integration of the latest machine and deep learning-based methods being work in progress. Unsupervised matching consists of two steps:

1. The similarity estimation between all candidate pairs generated by blocking. Java JedAI leverages a series of established similarities based on TF, TF-IDF and Boolean weights of sparse vectors whose dimensions correspond to token or character n-grams ($n=1, 2$ or 3 for the former and $2, 3, 4, 5$ or 6 for the latter).
2. The clustering of the weighted candidate pairs according to an unconstrained algorithm, i.e., an algorithm that does not require the number of duplicates as input, while being capable of generating single clusters, i.e., clusters containing a single entity that has no duplicates. For Dirty ER, JedAI implements all methods examined in [27], while for Clean-Clean ER, it implements three methods:
 - a. *Unique Mapping Clustering* [28], which sorts all pairwise comparisons in decreasing similarity score and iteratively considers the top one as a match, if its score exceeds a predetermined threshold and none of the involved profiles has already been matched.
 - b. *Row-Column Clustering*, which implements an efficient approximation of the Hungarian Algorithm [29].
 - c. *Best Assignment Clustering*, which implements an efficient, heuristic solution to the assignment problem in unbalanced bipartite graphs [30].

Below, we explain how pyJedAI improved both aspects of JedAI's unsupervised matching.

4.2.1 Enhancing the similarity estimation

As explained above, Java JedAI exclusively supports the *syntactic* similarity between entity profiles. There is a large variety of weighting schemes, tokenization methods and similarity functions, but all possible combination are restricted to the textual information that is shared by two entity profiles.

All these methods are also supported by pyJedAI. However, pyJedAI goes beyond them [34] by supporting *semantic* similarities, as well. This is achieved by vectorizing the schema-agnostic sentence that represents every entity profile using any of the language models in Table 3. The resulting vectors are converted into similarity values with one of the following functions:

- 1) Cosine similarity of the two embedded vectors.
- 2) Euclidean similarity, which is estimated as $1/(1+\text{Euclidean distance})$.
- 3) Word mover's similarity, which amounts to $1/(1+\text{Word mover's distance})$.

The extensive experimental analysis in [34] estimated the maximum F-measure that corresponds to each language model when combined with the *Unique Mapping Clustering* algorithm. Only the similarity threshold is fine-tuned, determining the lowest similarity of the candidate pairs that are considered as candidate matches (i.e., pairs with a lower similarity score are disregarded). The experimental results indicate that S-GTR-T5 consistently outperforms all other language models.

This model can be applied to a set of candidate pairs through the following commands:

```
from pyjedai.clustering import UniqueMappingClustering

ccc = UniqueMappingClustering()
clusters = ccc.process(g, data, similarity_threshold=0.5)
```

where *g* is the graph returned as second argument by *build_blocks*.

4.2.2 Enhancing the clustering for Clean-Clean ER

To boost the performance of pyJedAI's unsupervised matching in the case of Clean-Clean ER, we searched the literature for bipartite graph matching algorithms that satisfy the following criteria:

1. They are learning-free, requiring no labelled dataset for partitioning the similarity graph into a set of equivalence clusters (each equivalence cluster contains all entity profiles describing the same real-world object).
2. Their time complexity is lower than the quadratic one of the brute-force approach.
3. Their space complexity is linear with respect to the number of nodes (i.e., entity profiles) and edges (i.e., candidate pairs).

Apart from the three clustering algorithms mentioned in Section 4.2, the following algorithms satisfy these criteria, too:

1. *Connected Components*, which simply computes the transitive closure of the similarity and considers as equivalence clusters those containing only two entities, one from each partition.
2. *Ricochet Sequential Rippling Clustering*, which adapts the homonymous method for Dirty ER to Clean-Clean ER simply by considering as equivalence clusters those containing just one record from each input data set.
3. *Best Match Clustering*, which solves the Stable Marriage problem. It selects one of the two graph partitions and for each one of its nodes, it retains as match the top-weighted edge that so far has not been matched to another entity.
4. *Exact Clustering*, which considers as match only the nodes from different partitions that are the top-weighted neighbors of each other.
5. *Király's Clustering*, which is an adaptation of the linear time $3/2$ approximation to the Maximum Stable Marriage problem.

For all algorithms, the sole configuration parameter is the similarity threshold, below which the edges are discarded. This edge pruning is applied as a pre-processing step.

To assess the relative performance of the 8 bipartite graph clustering algorithms implemented by pyJedAI, we performed an extensive experimental analysis [36] on 769 similarity graphs stemming from 10 real-world datasets in combination with both schema-agnostic and schema-based settings as well as with both syntactic and semantic similarities. The latter combine fastText and S-GTR-T5 embedding vectors with the three similarity measures defined in Section 4.2, whereas the former rely on character and token n-grams similarities. Each algorithm was fine-tuned with respect its similarity threshold based on the ground-truth. The experimental results suggest that:

- The most effective algorithm for a particular similarity graph mainly depends on the type of edge weights and the portion of duplicates with respect to the total number of nodes/records.

STELAR – D3.1: Schema matching and entity linking (v1)

- Unique Mapping Clustering is typically the top performer in *balanced datasets*, where the vast majority of entity profiles in the one dataset matches with a record of the other dataset and vice versa.
- Király's Clustering is the top performer, followed in close distance by Exact Clustering, in all other datasets. These include the *scarce datasets*, where only a small portion of the entity profiles in both data sources has a duplicate, and *one-sided datasets*, where most entities from one data source have duplicates, unlike the other data source.

5. New functionalities for Geospatial Interlinking

In this section, we delve into the outcomes of Task 3.2 that go beyond the state-of-the-art in Geospatial Interlinking.

5.1 pyJedAI-spatial

pyJedAI-spatial [37] is a specialized component of pyJedAI that focuses on geospatial entities, identifying the topological relations between them. It actually constitutes a library of the state-of-the-art techniques for Geospatial Interlinking, organizing the relevant algorithms into a novel taxonomy formed by three dimensions, as shown in Figure 10:

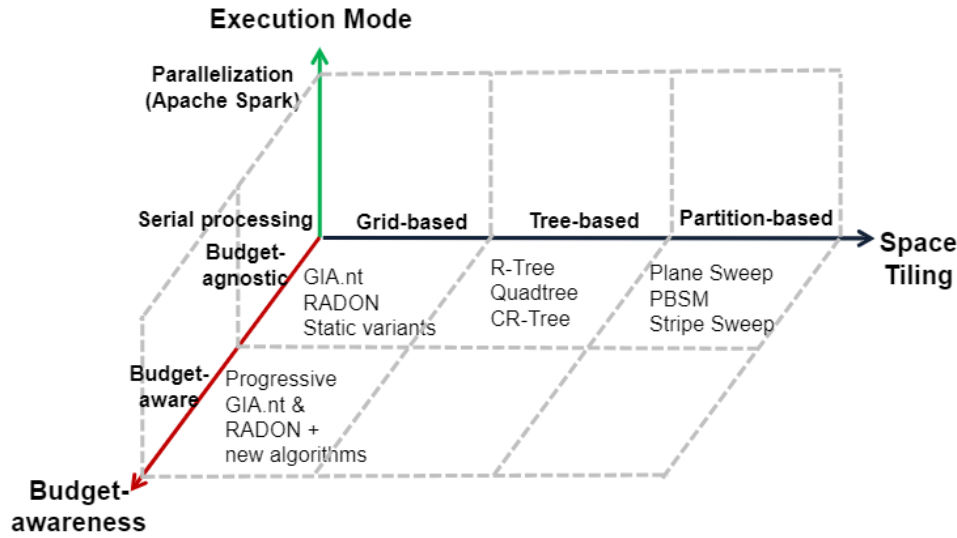


Figure 10. The solution space of Geospatial Interlinking algorithms that can be constructed by pyJedAI-spatial.

1. *Space Tiling* distinguishes the Geospatial Interlinking algorithms into three categories:
 - a. The *grid-based algorithms* are Semantic Web techniques that define a static or dynamic Equigrid, i.e., RADON, GIA.nt and their static, data independent variants.
 - b. The *tree-based algorithms* are main-memory spatial join techniques from the database community, i.e., R-Tree, Quadtree and CR-Tree.
 - c. The *partition-based algorithms* are computational geometry techniques, namely Plane Sweep, PBSM and a custom adaptation called Stripe Sweep.
2. *Budget-awareness* categorizes algorithms into two categories.
 - a. The *budget-agnostic* methods executed in a batch manner that processes the input data in no particular order and produces results only upon completion of the entire process. These include all grid-, tree- and partition-based algorithms mentioned above.
 - b. The *budget-aware methods* are suitable for applications with limited computational or temporal resources, producing results progressively, in a pay-as-you-go manner. These include Progressive RADON, Progressive GIA.nt and Dynamic Progressive GIA.nt.
3. *Execution mode* distinguishes between serialized algorithms, which run on a single CPU core, and massively parallel ones, which run on top of pySpark. The latter is work in progress.

pyJedAI-spatial creates any end-to-end pipeline that is defined by these three dimensions. Special care is taken to ensure high time efficiency and scalability. To this end, the implementation of every supported algorithm relies on custom improvements that reduce memory requirements or unnecessary computations. For example, a numeric id is used as a reference to every geometry, instead of its URI, while the reference point technique [26] is used in all algorithms to efficiently avoid repeated verifications of the same geometries in different tiles.

Note also that pyJedAI-spatial supports a broad range of data formats. Through its Data Reading component, it handles the most popular structured and semi-structured formats that are used for

encoding geometries: Well Known Text (WKT) files, CSV and TSV files, GeoJSON files, RDF dumps, JsonRDF as well as SPARQL endpoints. In this way, JedAI-spatial is able to interlink heterogeneous datasets, e.g., WKT data with GeoJSON files.

5.2 Supervised Progressive GIA.nt

This approach, detailed in [38], goes beyond Progressive GIA.nt by replacing its learning-free Scheduling, which is based on heuristics, with a principled, supervised approach that trains a probabilistic classifier on-the-fly, without requiring any labelling by the user, and associates every pair with a probability that its constituent geometries are topologically related.

More specifically, Supervised Progressive GIA.nt operates as follows:

- It performs the same Filtering as (Progressive) GIA.nt, building an Equigrid, whose dimensions correspond to the average length and width of the source geometries.
- A first pass over the target geometries is performed to estimate important statistics and to sample the candidate geometry pairs that will be automatically labelled (through the exact Verification of Geospatial Interlinking) to create the training set. There are two requirements for sampling:
 - (i) The selected pairs should have intersecting MBRs, because pairs with disjoint MBRs are ignored in Geospatial Interlinking.
 - (ii) The selected pairs should be representative of all geometry pairs with intersecting MBRs.

To address the heavy class imbalance (in favor of disjoint geometry pairs), undersampling is used: the number of positive and negative pairs in the training set is equal. This means that the excessive negative/disjoint pairs that are automatically labelled in this step are ignored.

- The probabilistic classifier is trained on the resulting labelled dataset. Any established machine learning algorithm can be used in this process, such as Logistic Regression, Naïve Bayes or Random Forests. The only requirement is that it provides a probabilistic estimation to each pair.
- The second pass over the target geometries is performed, applying the learned classifier to every pair of geometries with intersecting MBRs. These pairs are sorted in decreasing classification probability and the N top-weighted ones are verified to extract the topological relations they satisfy (if any).

To ensure high effectiveness, Supervised Progressive GIA.nt employs 31 features that capture all relevant evidence for a pair of geometries. They are distinguished into four complementary categories:

- (i) The *area-based features* consider the space occupied by the MBR of each geometry.
- (ii) The *boundary-based features* stem from the characteristics of each geometry's boundary.
- (iii) The *grid-based features* emanate from Filtering's index.
- (iv) The *candidate-based features* rely on the candidates associated with every geometry after Filtering.

The first two categories depend exclusively on the characteristics of the geometries, unlike the other two categories that rely on Filtering. Every category of feature is further distinguished into two types:

- (i) The *atomic features* pertain to core characteristics of a single geometry.
- (ii) The *composite features* combine multiple atomic features into a single score that is typically normalized in $[0, 1]$, with higher values implying a stronger likelihood for topological relatedness.

Table 4 categorizes the 31 proposed classification features according to the two-dimensional taxonomy that is formed by the above categories.

Table 4. The features defined in [38] for Supervised Progressive GIA.nt.

	Atomic	Composite
Area-based	F1. source MBR area F2. target MBR area F3. intersection MBR area	F4. intersection MBR normalized by source MBR = $F3/F1$ F5. intersection MBR normalized by target MBR = $F3/F2$ F6. Jaccard MBR overlap = $F3/(F1+F2-F3)$
Boundary-based	F7. number of source boundary points F8. number of target boundary points F9. source boundary length	F11. normalized source boundary complexity = $F7/F9$ F12. normalized target boundary complexity = $F8/F10$

	F10. target boundary length	
φGrid-based	F13. number of tiles intersecting the source MBR F14. number of tiles intersecting the target MBR F15. number of common tiles	F16. common tiles normalized by source tiles = $F15/F13$ F17. common tiles normalized by target tiles = $F15/F14$ F18. Jaccard common tiles = $F15/(F13+F14-F15)$ F19. Pearson's χ^2 test
Candidate-based	F20. total candidates for source geometry F21. distinct candidates for source geometry F22. real candidates for source geometry F23. total candidates for target geometry F24. distinct candidates for target geometry F25. real candidates for target geometry	F26. source distinct candidates normalized by total = $F21/F20$ F27. source real candidates normalized by total = $F22/F20$ F28. source real candidates normalized by distinct = $F22/F21$ F29. target distinct candidates normalized by total = $F24/F23$ F30. target real candidates normalized by total = $F25/F23$ F31. target real candidates normalized by distinct = $F25/F24$

After a thorough feature selection carried out on large, real-world datasets, the combination of all atomic features is selected as the best approach. The sampling is configured so that it labels just 500 pairs from each category, while the best performing classification algorithm is Logistic Regression. The resulting configuration of Supervised Progressive GIA.nt consistently outperforms any configuration of the original, learning-free Progressive GIA.nt.

6. Conclusions and Next Steps

6.1. Conclusions

This deliverable presented pyJedAI, a new system for Schema Matching and Entity Linking, which can be used in a large variety of academic and industrial applications, especially those leveraging Python’s open-source data science ecosystem. pyJedAI is part of the KLMS Tools, responsible for data interlinking. It offers a library of state-of-the-art techniques for Schema Matching, Entity Resolution (a generalization of Entity Linking) and Geospatial Interlinking (a specialization of Entity Linking). For each of these tasks, we reported the existing methods that have been integrated into pyJedAI, either as Python packages or through custom implementations. Most importantly, we highlighted the advancements beyond the state-of-the-art in each of these tasks during STELAR, based on the research works that have already been published.

All considered methods belong to the *first generation*, which target Veracity in the context of clean schemata with low levels of noise, or to the *third generation*, which target Veracity, Volume and Variety in the context of high heterogeneity and levels of noise in both attribute names and values. As a result, they already meet the technical requirements R9 (Entity Linking/T3.2), R12 (Schema Matching/R3.1) and R23 (Geospatial Interlinking/T3.2) specified in D1.1 (Table 15). Thus, pyJedAI can already be integrated into the use case scenarios A.4 and A.6 specified in D5.1.

Further improvements are currently ongoing to cover the second and fourth generations, too, as explained below in more detail, so as to enhance pyJedAI’s performance when meeting these requirements and/or its scope of when incorporated into these use case scenarios.

6.2. Next steps

This section elaborates on the extensions of pyJedAI that are planned to be completed by M18, which corresponds to Milestone 3, when the initial version of the STELAR KLMS will be released.

For **Schema Matching**, there are three directions of immediate extensions:

1. The integration of Large Language Models as a novel technique for Schema Matching, based on Retrieval-augmented Generation for higher effectiveness (i.e., fewer hallucinations). The initial experiments are quite encouraging.
2. The creation of heterogeneous datasets involving large schemata and/or ontologies that can be combined with Entity Resolution, to highlight the interdependence of the two tasks. Ontologies from the food sector will be considered as well as the large sample of owl:sameAs statements provided by <https://www.sameas.cc>.
3. The incorporation of LogMap ideas for high scalability in the methods already supported by pyJedAI for schema matching and schema clustering.

For **Entity Resolution**, an extension applying to both Filtering and Verification is the incorporation of *second-generation techniques* through parallelization. Two are the main approaches that are currently considered:

- Multicore parallelization with Ray²⁴
- Massive parallelization with pySpark.

In both cases, parallelization will be considered only for methods that exhibit (almost) linear scalability with the increase in the size of the data and the number of cores or nodes. That is, quite efficient methods with low overhead over a single CPU core won’t be parallelized.

For **Filtering**, the immediate goal is to fully integrate *TokenJoin* into PyJedAI as a similarity join algorithm along with a recent, state-of-the-art approach based on TF-IDF joins, *Sparkly* [31]. We are also developing a novel, stochastic approach that excels in scalability compared to all other filtering

²⁴ <https://github.com/ray-project/ray>

methods, called *Random Join*: first, it applies a fine-grained blocking workflow, consisting of Token Blocking, Block Purging and Block Filtering (with default configuration), and then, it randomly selects pairs inside the resulting blocks.

For **Verification**, we are extending pyJedAI so that it supports all state-of-the-art DL-based matching algorithms. Through a complex architecture that leverages Docker²⁵, Redis²⁶ and BullMQ²⁷, pyJedAI implements an end-to-end pipeline that applies a Filtering technique, automatically labels the resulting candidate pairs based on the ground-truth of the given datasets, and trains and tests any of the supported open-source DL-based matchers. This pipeline, the first of its kind in the literature, has the following benefits:

1. It allows for benchmarking their relative performance in terms of effectiveness, time efficiency and memory requirements in more realistic settings than the few fixed datasets that are typically used in the literature.
2. It allows for examining the impact of Filtering on DL-based Verification, a topic that has been overlooked in the literature.
3. It allows for juxtaposing DL-based with LLM-based Verification, which will also be integrated into pyJedAI.

Finally, for **Geospatial Interlinking**, pyJedAI will be extended with *Proactive Geospatial Interlinking*. Based on Supervised Progressive GIA.nt, it uses the sampled set of labelled geometry pairs to train a kernel density estimator, which provides the minimum classification probability that satisfies a user-defined recall level. By discarding all geometry pairs with a lower probability, Geospatial Interlinking minimizes the Verification cost for the data and performance requirements at hand.

Finally, for usability and KLMS integration, pyJedAI needs to be containerized and served as a service. The development of an Application Programming Interface (API) is recommended to cater to STELAR’s requirements, ensuring seamless on-demand access to pyJedAI functionalities.

6.3. Long-term goals

In this section, we briefly discuss the extensions in pyJedAI that will be carried out in the second half of STELAR, between Milestone 3 on M18 and the end of Tasks 3.1 and 3.2 on M33.

The first long-term goal is to incorporate fourth-generation methods that support two kinds of applications:

1. Those with limited temporal and/or computational resources that can accommodate partial results. For these applications, pyJedAI will implement progressive techniques that gradually produce results in a pay-as-you manner.
2. Those with strict requirements in terms of response time. For these applications, pyJedAI will implement methods for real-time and query-driven ER.

The second long-term goal is to maximize the usability of pyJedAI through automatic configuration of end-to-end pipelines [39]. Given that every method in every component of pyJedAI’s workflows involves at least one configuration parameter, the fine-tuning of a complex pipeline that involves multiple methods is a non-trivial task, even for expert users. To ameliorate this issue, pyJedAI will incorporate existing and novel techniques that support configuration optimization in the following scenarios:

1. *Known ground-truth and pipeline*, where the goal is to fine-tune a specific end-to-end workflow over a given dataset, which is accompanied by the golden standard of real duplicates.
2. *Known ground-truth, but unknown pipeline*, where the goal is to identify the best workflow for the data at hand among those supported by pyJedAI, based on its golden standard of duplicates.

²⁵ <https://www.docker.com>

²⁶ <https://redis.io>

²⁷ <https://docs.bullmq.io>

3. *Unknown ground-truth, but known pipeline*, where the goal is to fine-tune a specific pipeline that is expected to yield high performance to a dataset that lacks a golden standard of duplicates.
4. *Unknown ground-truth and pipeline*, where the goal is to recommend configurations in the realistic settings where the user has not selected any workflow and the dataset lacks a golden standard of duplicates.
5. *Type-specific configurations*, where the goal is to automatically partition the entity profiles in the KLMS according to their type and then propose the best end-to-end workflow per type.

References

- [1] Erhard Rahm, Philip A. Bernstein. A survey of approaches to automatic schema matching. VLDB J. 10(4): 334-350 (2001)
- [2] Christos Koutras, George Siachamis, Andra Ionescu, Kyriakos Psarakis, Jerry Brons, Marios Fragkoulis, Christoph Lofi, Angela Bonifati, Asterios Katsifodimos. Valentine: Evaluating Matching Techniques for Dataset Discovery. ICDE 2021: 468-479
- [3] Sergey Melnik, Hector Garcia-Molina, Erhard Rahm. Similarity Flooding: A Versatile Graph Matching Algorithm and Its Application to Schema Matching. ICDE 2002: 117-128
- [4] Jayant Madhavan, Philip A. Bernstein, Erhard Rahm. Generic Schema Matching with Cupid. VLDB 2001: 49-58
- [5] Hong Hai Do, Erhard Rahm. COMA - A System for Flexible Combination of Schema Matching Approaches. VLDB 2002: 610-621
- [6] Riccardo Cappuzzo, Paolo Papotti, Saravanan Thirumuruganathan. Creating Embeddings of Heterogeneous Relational Datasets for Data Integration Tasks. SIGMOD Conference 2020: 1335-1349
- [7] Raul Castro Fernandez, Essam Mansour, Abdulhakim Ali Qahtan, Ahmed K. Elmagarmid, Ihab F. Ilyas, Samuel Madden, Mourad Ouazzani, Michael Stonebraker, Nan Tang. Seeping Semantics: Linking Datasets Using Word Embeddings for Data Discovery. ICDE 2018: 989-1000
- [8] Ernesto Jiménez-Ruiz, Bernardo Cuenca Grau, LogMap: Logic-Based and Scalable Ontology Matching. ISWC (1) 2011: 273-288
- [9] Yuan He, Jiaoyan Chen, Denvar Antonyrajah, Ian Horrocks. BERTMap: A BERT-Based Ontology Alignment System. AAAI 2022: 5684-5691
- [10] Peter Christen. Data Matching - Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection. Data-Centric Systems and Applications, Springer 2012.
- [11] Xin Luna Dong, Divesh Srivastava. Big Data Integration. Synthesis Lectures on Data Management, Morgan & Claypool Publishers 2015.
- [12] Vassilis Christophides, Vasilis Efthymiou, Kostas Stefanidis. Entity Resolution in the Web of Data. Synthesis Lectures on the Semantic Web: Theory and Technology, Morgan & Claypool Publishers 2015.s
- [13] George Papadakis, Ekaterini Ioannou, Emanouil Thanos, Themis Palpanas. The Four Generations of Entity Resolution. Synthesis Lectures on Data Management, Morgan & Claypool Publishers 202.
- [14] Lise Getoor, Ashwin Machanavajjhala. Entity Resolution: Theory, Practice & Open Challenges. Proc. VLDB Endow. 5(12): 2018-2019 (2012)
- [15] Kostas Stefanidis, Vasilis Efthymiou, Melanie Herschel, Vassilis Christophides. Entity resolution in the web of data. WWW (Companion Volume) 2014: 203-204
- [16] George Papadakis, Ekaterini Ioannou, Themis Palpanas. Entity Resolution: Past, Present and Yet-to-Come. EDBT 2020: 647-650
- [17] Peter Christen. A Survey of Indexing Techniques for Scalable Record Linkage and Deduplication. IEEE Trans. Knowl. Data Eng. 24(9): 1537-1555 (2012)
- [18] Vassilis Christophides, Vasilis Efthymiou, Themis Palpanas, George Papadakis, Kostas Stefanidis. An Overview of End-to-End Entity Resolution for Big Data. ACM Comput. Surv. 53(6): 127:1-127:42 (2021)
- [19] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, Themis Palpanas. Blocking and Filtering Techniques for Entity Resolution: A Survey. ACM Comput. Surv. 53(2): 31:1-31:42 (2021)

- [20] Panayiotis Smeros and Manolis Koubarakis. 2016. Discovering Spatial and Temporal Links among RDF Data. In Workshop on Linked Data on the Web (LDOW2016).
- [21] Mohamed Ahmed Sherif, Kevin Dreßler, Panayiotis Smeros, Axel-Cyrille Ngonga Ngomo. Radon - Rapid Discovery of Topological Relations. AAAI 2017: 175-181
- [22] Abdullah Fathi Ahmed, Mohamed Ahmed Sherif, Axel-Cyrille Ngonga Ngomo. RADON2 - a buffered-intersection matrix computing approach to accelerate link discovery over geo-spatial RDF knowledge bases: OAEI2018 results. OM@ISWC 2018: 197-204
- [23] Georgios M. Santipantakis, Apostolos Glenis, Christos Doulkeridis, Akrivi Vlachou, George A. Vouros. stLD: towards a spatio-temporal link discovery framework. SBD@SIGMOD 2019: 4:1-4:6
- [24] George Papadakis, Georgios M. Mandilaras, Nikos Mamoulis, Manolis Koubarakis. Progressive, Holistic Geospatial Interlinking. WWW 2021: 833-844
- [25] George Papadakis, George Mandilaras, Nikos Mamoulis, Manolis Koubarakis. Static and Dynamic Progressive Geospatial Interlinking. ACM Trans. Spatial Algorithms Syst. 8(2): 1-41 (2022)
- [26] Jens-Peter Dittrich, Bernhard Seeger. Data Redundancy and Duplicate Detection in Spatial Join Processing. ICDE 2000: 535-546
- [27] Oktie Hassanzadeh, Fei Chiang, Renée J. Miller, Hyun Chul Lee. Framework for Evaluating Clustering Algorithms in Duplicate Detection. Proc. VLDB Endow. 2(1): 1282-1293 (2009)
- [28] Simon Lacoste-Julien, Konstantina Palla, Alex Davies, Gjergji Kasneci, Thore Graepel, Zoubin Ghahramani. SIGMa: simple greedy matching for aligning large knowledge bases. KDD 2013: 572-580
- [29] Harold W. Kuhn. The Hungarian Method for the Assignment Problem. 50 Years of Integer Programming 2010: 29-47
- [30] L. Ramshaw, R.E. Tarjan, On Minimum-Cost Assignments in Unbalanced Bipartite Graphs, Tech. Rep. HPL-2012-40R1, HP Labs, Palo Alto, CA, USA,
- [31] Derek Paulsen, Yash Govind, AnHai Doan. Sparkly: A Simple yet Surprisingly Strong TF/IDF Blocker for Entity Matching. Proc. VLDB Endow. 16(6): 1507-1519 (2023)
- [32] Konstantinos Nikoletos, George Papadakis, Manolis Koubarakis. pyJedAI: a Lightsaber for Link Discovery. ISWC (Posters/Demos/Industry) 2022. Available at: <https://zenodo.org/records/10209826>
- [33] George Papadakis, Marco Fisichella, Franziska Schoger, George Mandilaras, Nikolaus Augsten, Wolfgang Nejdl. Benchmarking Filtering Techniques for Entity Resolution. ICDE 2023: 653-666. Available at: <https://zenodo.org/records/8084969>
- [34] Alexandros Zeakis, George Papadakis, Dimitrios Skoutas, Manolis Koubarakis. Pre-trained Embeddings for Entity Resolution: An Experimental Analysis. Proc. VLDB Endow. 16(9): 2225-2238 (2023). Available at: <https://zenodo.org/records/8083349>
- [35] Alexandros Zeakis, Dimitrios Skoutas, Dimitris Sacharidis, Odysseas Papapetrou, Manolis Koubarakis. TokenJoin: Efficient Filtering for Set Similarity Join with Maximum Weighted Bipartite Matching. Proc. VLDB Endow. 16(4): 790-802 (2022). Available at: <https://zenodo.org/records/7625991>
- [36] George Papadakis, Vasilis Efthymiou, Emmanouil Thanos, Oktie Hassanzadeh, Peter Christen. An analysis of one-to-one matching algorithms for entity resolution. VLDB J. 32(6): 1369-1400 (2023). Available at: <https://zenodo.org/records/7844529>
- [37] Marios Papamichalopoulos, George Papadakis, George Mandilaras, Maria Despoina Siampou, Nikos Mamoulis, Manolis Koubarakis. Three-dimensional Geospatial Interlinking with JedAI-spatial. Journal of Web Semantics (under minor revision). Available at: <https://arxiv.org/pdf/2205.01905.pdf>

[38] Maria Despoina Siampou, George Papadakis, Nikos Mamoulis, Manolis Koubarakis. Supervised Scheduling for Geospatial Interlinking. ACM SIGSPATIAL 2023. Available at: <https://zenodo.org/records/10070449>

[39] Vasilis Efthymiou, Ekaterini Ioannou, Manos Karvounis, Manolis Koubarakis, Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, Dimitris Skoutas, Yannis Velegrakis, Alexandros Zeakis. Self-configured Entity Resolution with pyJedAI. 2023 IEEE International Conference on Big Data (vision paper). Available at: <https://zenodo.org/records/10070414>